

La librería que buscas es **ExcelGen**, creada por **Marc Bleron** y disponible en GitHub. Es una utilidad PL/SQL de código abierto (licencia MIT) que permite generar archivos Excel nativos (.xlsx y .xlsb) directamente desde Oracle, a partir de consultas SQL o cursores, sin necesidad de Java ni de software externo. Se mantiene activamente (la última actualización fue hace pocas semanas) y es ideal para tu entorno de PL/SQL Developer y Oracle Forms.

A continuación, tienes la guía paso a paso para instalarla y usarla.

Paso 1: Descargar e instalar ExcelGen

ExcelGen depende de dos submódulos: **ExcelCommons** y **MSUtilities**. La forma más limpia de obtener todo es clonar el repositorio con la opción `--recurse-submodules`.

Opción A: Si tienes Git instalado

Abre una consola (CMD o Git Bash) y ejecuta:

```
bash
```

```
git clone --recurse-submodules https://github.com/mbleron/ExcelGen.git
```

Esto creará una carpeta `ExcelGen` con todo lo necesario.

Opción B: Si no tienes Git

1.Descarga el ZIP del repositorio:

<https://github.com/mbleron/ExcelGen/archive/refs/heads/master.zip>

2.Descarga también los ZIP de los submódulos:

[ExcelCommons](#)

[MSUtilities](#)

3.Descomprime los tres en una misma carpeta, de modo que queden así:

```
text
```

```
ExcelGen/
├─ ExcelCommons/      (contenido del ZIP de ExcelCommons)
├─ MSUtilities/       (contenido del ZIP de MSUtilities)
├─ install.sql
├─ install_nocrypto.sql
└─ ... (resto de archivos)
```

Instalar en la base de datos

Conéctate con **PL/SQL Developer** (o SQL*Plus) como el usuario donde quieras instalar la librería (por ejemplo, tu esquema de desarrollo) y ejecuta el script de instalación:

```
sql
```

```
@install.sql
```

Si tu usuario no tiene permisos sobre `DBMS_CRYPTO` (lo cual es común), ejecuta en su lugar:

```
sql
```

```
@install_nocrypto.sql
```

⚠ **Nota:** Si usas `install_nocrypto.sql`, no podrás usar la función de encriptación de archivos, pero el resto de funcionalidades (incluyendo la generación de Excel a partir de queries) funcionarán perfectamente.



Paso 2: Crear un Excel desde una consulta SQL (ejemplo básico)

El flujo típico con ExcelGen es:

1. **Crear un contexto** (`createContext`).
2. **Añadir una hoja** basada en una consulta (`addSheetFromQuery`).
3. **Configurar encabezados** (opcional pero recomendado).
4. **Generar el archivo** (`createFile`) → devuelve un `BLOB`.
5. **Cerrar el contexto** (`closeContext`).

Ejemplo: exportar los puntos de venta en mora

Supongamos que quieres exportar todos los puntos de venta de la división "Clientes de Mi Negocio" que están en estado `INACTIVO` (bloqueados por mora). La consulta sería:

sql

```
SELECT pv.id_punto_venta,
       pv.nombre_pv,
       pv.estado,
       a.nombre_agencia,
       t.nombre_territorio
FROM punto_venta pv
JOIN agencia a ON a.id_agencia = pv.id_agencia
JOIN territorio t ON t.id_territorio = a.id_territorio
JOIN division d ON d.id_division = t.id_division
WHERE d.nombre_division = 'Clientes de Mi Negocio'
      AND pv.estado = 'INACTIVO';
```

El bloque PL/SQL para generar el Excel y obtener el `BLOB` sería:

sql

```
DECLARE
    v_ctx_id      ExcelGen.ctxHandle;
    v_sheet_id    ExcelGen.sheetHandle;
    v_excel_blob  BLOB;
BEGIN
    -- 1. Crear el contexto (formato XLSX por defecto)
    v_ctx_id := ExcelGen.createContext();

    -- 2. Añadir una hoja basada en la consulta SQL
    v_sheet_id := ExcelGen.addSheetFromQuery(
        p_ctxId      => v_ctx_id,
        p_sheetName => 'PV Bloqueados',
        p_query      => 'SELECT pv.id_punto_venta, pv.nombre_pv, pv.estado, ' ||
                        '      a.nombre_agencia, t.nombre_territorio ' ||
                        ' FROM punto_venta pv ' ||
                        ' JOIN agencia a ON a.id_agencia = pv.id_agencia ' ||
                        ' JOIN territorio t ON t.id_territorio = a.id_territorio ' ||
                        ' JOIN division d ON d.id_division = t.id_division ' ||
                        ' WHERE d.nombre_division = ''Clientes de Mi Negocio'' ' ||
```

```

        '        AND pv.estado = ''INACTIVO'''
    );

-- 3. Activar la fila de encabezado y congelarla (opcional)
ExcelGen.setHeader(
    p_ctxId    => v_ctx_id,
    p_sheetId => v_sheet_id,
    p_frozen   => TRUE
);

-- 4. Generar el archivo y obtener el BLOB
ExcelGen.createFile(
    p_ctxId    => v_ctx_id,
    p_directory => 'TU_DIRECTORY', -- Un DIRECTORY de Oracle
    p_filename  => 'pv_bloqueados.xlsx',
    p_blob      => v_excel_blob      -- Aquí se devuelve el BLOB
);

-- 5. Cerrar el contexto
ExcelGen.closeContext(v_ctx_id);

-- En este punto, v_excel_blob contiene el archivo Excel listo
DBMS_OUTPUT.PUT_LINE('Excel generado. Tamaño: ' || DBMS_LOB.GETLENGTH(v_excel_blob) || '
bytes');
END;
/

```

Importante sobre p_directory: createFile puede escribir el archivo en un DIRECTORY de Oracle, pero **también** devuelve el BLOB a través del parámetro p_blob. Si solo necesitas el BLOB (por ejemplo, para adjuntarlo a un correo), puedes pasar un directorio cualquiera y quedarte con el BLOB. También existe una sobrecarga que **solo devuelve el BLOB** sin escribir en disco (consulta la documentación de createFile en el README para más detalles).

✉ Paso 3: Adjuntar el Excel generado a un correo

Ahora que tienes el BLOB en la variable v_excel_blob, puedes usar APEX_MAIL (o UTL_MAIL) para enviarlo como adjunto. El procedimiento completo sería:

sql

```
CREATE OR REPLACE PROCEDURE pr_enviar_excel_pv_mora
```

```
IS
```

```
    v_ctx_id      ExcelGen.ctxHandle;  
    v_sheet_id    ExcelGen.sheetHandle;  
    v_excel_blob  BLOB;  
    v_mail_id     NUMBER;
```

```
BEGIN
```

```
    -- Generar el Excel (mismo código del paso anterior)
```

```
    v_ctx_id := ExcelGen.createContext();
```

```
    v_sheet_id := ExcelGen.addSheetFromQuery(  
        v_ctx_id,
```

```
        'PV Bloqueados',  
        'SELECT pv.id_punto_venta, pv.nombre_pv, pv.estado, a.nombre_agencia ' ||  
        ' FROM punto_venta pv ' ||  
        ' JOIN agencia a ON a.id_agencia = pv.id_agencia ' ||  
        ' JOIN territorio t ON t.id_territorio = a.id_territorio ' ||  
        ' JOIN division d ON d.id_division = t.id_division ' ||  
        ' WHERE d.nombre_division = ''Clientes de Mi Negocio'' ' ||  
        ' AND pv.estado = ''INACTIVO'''
```

```
    );
```

```
    ExcelGen.setHeader(v_ctx_id, v_sheet_id, p_frozen => TRUE);
```

```
    ExcelGen.createFile(v_ctx_id, 'TU_DIRECTORIO', 'pv_bloqueados.xlsx', v_excel_blob);
```

```
    ExcelGen.closeContext(v_ctx_id);
```

```
    -- Enviar el correo con el adjunto
```

```
    v_mail_id := APEX_MAIL.SEND(  
        p_to
```

```
            => 'cobranza1@empresa.com,cobranza2@empresa.com,gerencia@empresa.com',
```

```
        p_from
```

```
            => 'alertas@empresa.com',
```

```
        p_subj
```

```
            => 'Reporte de PV Bloqueados - ' || TO_CHAR(SYSDATE, 'DD/MM/YYYY'),
```

```
        p_body
```

```
            => 'Se adjunta el reporte de puntos de venta bloqueados.',
```

```
        p_body_html => '<html><body><p>Reporte de PV bloqueados adjunto.</p></body></html>'
```

```
    );
```

```
    APEX_MAIL.ADD_ATTACHMENT(  
        p_mail_id
```

```
            => v_mail_id,
```

```
        p_attachment => v_excel_blob,
```

```
        p_filename
```

```
            => 'pv_bloqueados.xlsx',
```

```
        p_mime_type => 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
```

```
    );
```

```
APEX_MAIL.PUSH_QUEUE;

DBMS_OUTPUT.PUT_LINE('Correo enviado con Excel adjunto.');
```

EXCEPTION

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        RAISE;
END pr_enviar_excel_pv_mora;
/
```



Paso 4: Personalización adicional (opcional)

ExcelGen permite mucho más que un simple volcado de datos:

- **Múltiples hojas:** llamando a `addSheetFromQuery` varias veces con distintos nombres y consultas.
- **Formato de celdas:** usar `setRangeStyle`, `setNumFormat`, `setDateFormat`, etc.
- **Encabezados personalizados:** `setHeader` con parámetros para congelar filas, añadir filtros automáticos.
- **Validaciones y formato condicional:** funciones como `addDataValidationRule` o `addCondFormattingRule`.
- **Fórmulas:** `putFormulaCell`.
- **Imágenes:** `putImageCell` (desde la versión 4.4).

Para ver todas las posibilidades, consulta la sección "**ExcelGen Subprograms and Usage**" del README.



Consideraciones y alternativas

- **Permisos:** El usuario que ejecute ExcelGen necesita `EXECUTE` sobre el paquete `ExcelGen` y, si vas a usar `DBMS_CRYPTO`, los permisos correspondientes.
- **Directorio:** Para `createFile` necesitas un objeto `DIRECTORY` de Oracle con permisos de escritura. Si solo usas el BLOB, puedes pasar un directorio cualquiera (o usar la sobrecarga que no escribe en disco).

- **Alternativa ligera:** Si no quieres instalar ExcelGen, existe `as_xlsx` de Anton Scheffer, otra utilidad PL/SQL para generar XLSX. Es más sencilla pero menos potente (no soporta imágenes, validaciones, etc.).

Recomendación: ExcelGen es la opción más completa y activa. Con los pasos anteriores ya tienes todo para generar tus reportes en Excel y adjuntarlos a los correos automáticos que ya tienes planificados.

```

=====
=== -- Script PL/SQL para generar un Excel (ExcelGen) desde una query SQL y -- enviarlo por correo
electrónico (APEX_MAIL) como archivo adjunto. -- Requiere: ExcelGen instalado en la base de datos
(mbleron/ExcelGen). --
=====
===
--
=====
=== -- PASO 1: Crear el procedimiento que genera el Excel y envía el correo. --
=====
=== CREATE OR REPLACE PROCEDURE pr_enviar_excel_por_correo IS --
----- -- Variables para ExcelGen --
----- v_ctx_id ExcelGen.ctxHandle; --
Manejador del contexto del libro v_sheet_id ExcelGen.sheetHandle; -- Manejador de la hoja de
cálculo v_excel_blob BLOB; -- BLOB donde se almacenará el .xlsx
-----
-- Variables para el correo
-----
v_mail_id NUMBER; -- ID devuelto por APEX_MAIL.SEND
v_destinos VARCHAR2(500); -- Lista de correos separados por coma
v_remitente VARCHAR2(200); -- Correo del remitente
v_asunto VARCHAR2(300); -- Asunto del correo
v_nombre_archivo VARCHAR2(200); -- Nombre del archivo Excel adjunto
-----
-- Variable para la query SQL (debe devolver un cursor)
-----
v_query_sql VARCHAR2(32767);

BEGIN --
=====
CONFIGURACIÓN DE LA QUERY SQL -- Modifica esta consulta según tus necesidades. --
===== v
_query_sql := 'SELECT pv.id_punto_venta, ' || ' pv.nombre_pv, ' || ' pv.estado, ' || '

```

```

a.nombre_agencia, '||' t.nombre_territorio, '||' d.nombre_division, '||' c.nombre_cliente, '
||' COUNT(f.id_factura) AS facturas_vencidas, '||' MAX(TRUNC(SYSDATE -
f.fecha_vencimiento)) AS dias_mora, '||' SUM(f.saldo_pendiente) AS saldo_mora '||' FROM
punto_venta pv '||' JOIN agencia a ON a.id_agencia = pv.id_agencia '||' JOIN territorio t ON
t.id_territorio = a.id_territorio '||' JOIN division d ON d.id_division = t.id_division '||' JOIN
cliente c ON c.id_punto_venta = pv.id_punto_venta '||' JOIN servicio s ON s.id_cliente =
c.id_cliente '||' JOIN factura f ON f.id_servicio = s.id_servicio '||' WHERE d.nombre_division =
"Clientes de Mi Negocio" '||' AND f.estado IN ("VENCIDA", "EN_MORA") '||' AND
f.saldo_pendiente > 0 '||' GROUP BY pv.id_punto_venta, pv.nombre_pv, pv.estado, '||'
a.nombre_agencia, t.nombre_territorio, d.nombre_division, '||' c.nombre_cliente '||' ORDER BY
dias_mora DESC';

```

```

-- =====
-- GENERACIÓN DEL ARCHIVO EXCEL CON EXCELGEN
-- =====

-- 1. Crear el contexto (libro de Excel en memoria).
v_ctx_id := ExcelGen.createContext();

-- 2. Añadir una hoja a partir de la query SQL.
v_sheet_id := ExcelGen.addSheetFromQuery(
    p_ctxId      => v_ctx_id,
    p_sheetName => 'Mora PV',
    p_query      => v_query_sql
);

-- 3. Configurar la fila de encabezado (congelada y con autofiltro).
ExcelGen.setHeader(
    p_ctxId      => v_ctx_id,
    p_sheetId    => v_sheet_id,
    p_frozen     => TRUE
);

-- 4. Generar el archivo Excel y obtener su contenido como BLOB.
-- El parámetro p_directory puede ser cualquier DIRECTORY existente
-- (no se usará si solo se necesita el BLOB). Se pasa p_blob para
-- recibir el archivo generado.
ExcelGen.createFile(
    p_ctxId      => v_ctx_id,
    p_directory  => 'TMP_DIR',          -- Reemplazar por un DIRECTORY válido
    p_filename   => 'reporte_mora.xlsx',
    p_blob       => v_excel_blob
);

-- 5. Cerrar el contexto (liberar memoria).
ExcelGen.closeContext(v_ctx_id);

-- =====
-- ENVÍO DEL CORREO CON EL ARCHIVO ADJUNTO
-- =====

-- Configurar parámetros del correo.
v_destinos      :=
'cobranza1@empresa.com, cobranza2@empresa.com, gerencia@empresa.com';
v_remitente     := 'alertas@empresa.com';

```

```

v_asunto      := 'Reporte de Puntos de Venta en Mora - ' || TO_CHAR(SYSDATE,
'DD/MM/YYYY HH24:MI');
v_nombre_archivo := 'reporte_mora_' || TO_CHAR(SYSDATE, 'YYYYMMDD_HH24MI') ||
'.xlsx';

-- 1. Crear el mensaje de correo.
v_mail_id := APEX_MAIL.SEND(
    p_to      => v_destinos,
    p_from    => v_remitente,
    p_subj    => v_asunto,
    p_body    => 'Se adjunta el reporte de puntos de venta en mora.',
    p_body_html => '<html><body>' ||
                    '<p>Se adjunta el reporte de puntos de venta en mora.</p>' ||
                    '</body></html>'
);

-- 2. Adjuntar el archivo Excel generado (BLOB).
APEX_MAIL.ADD_ATTACHMENT(
    p_mail_id    => v_mail_id,
    p_attachment => v_excel_blob,
    p_filename   => v_nombre_archivo,
    p_mime_type  => 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet'
);

-- 3. Enviar el correo (empujar a la cola de salida).
APEX_MAIL.PUSH_QUEUE;

-- Mensaje de confirmación en la salida de PL/SQL Developer.
DBMS_OUTPUT.PUT_LINE('Correo enviado con adjunto: ' || v_nombre_archivo);

```

```

EXCEPTION WHEN OTHERS THEN -- En caso de error, mostrar el mensaje y relanzar la
excepción. DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM); RAISE; END
pr_enviar_excel_por_correo; /

```

```
--
```

```

=====
=== -- PASO 2: Ejecutar el procedimiento (bloque anónimo). --
=====

```

```

=== SET SERVEROUTPUT ON; BEGIN pr_enviar_excel_por_correo; END; /

```

